Document Number:	**P0671R1**, ISO/IEC JTC1 SC22 WG21
Audience:	EWG
Date:	2018-02-12
Author:	Axel Naumann (axel@cern.ch)

Parametric Functions

Table of Contents

- Don't think you had this discussion. Don't reject this without reading it.
- The Feature in a Nutshell
- Lots of Motivation
- Feature Details
- Acknowledgments
- Revision history
- References

Preamble

Please don't assume that you know what this paper is about. Several people have invested time e.g. for discussions of the feature, making it reasonable, and making it new (i.e. I am aware of past discussions). This is *different* and I appreciate that you make up your mind based on what's written here. I am almost certain: you haven't seen this before.

The Feature in a Nutshell

One of my colleagues' pet peeves is the lack of named parameters in C++ (a wonderful synopsis of pain is at [1]). It's seen as a usability limitation caused by a C syntax anachronism. With many current languages, passing arguments can be done using the parameter index (C-style) or its name.

So here is what I suggest: allow naming arguments in regular index-based, C++-style function calls, but ignoring them from the standard's point of view (but not necessarily so from the implementers'). An example:

The call is completely equivalent to this traditional one:

But if the call looks like this:

then implementations could warn that the parameter is known from one of the declarations of but is passed at the wrong position, and that isn't a parameter that shows up for any of the redeclarations ("maybe you've meant to call a different overload?"). Wouldn't that be an incredibly nice and helpful compiler!

Lots of Motivation

What we currently have on the call-site of functions is tuple-style: a series of types. What C++ advocates for is classes with named members. But in current C++, member names are used much less than function parameters. How come then that we still think "a parameter index is all a call could possibly hope for"?

There are multiple usage patterns where current actual code and humans dealing with it suffer, in reality, on a daily basis. Here they are (if you know more, let me know!)

Readability is maintainability

We all have functions that take three strings, or three ints, or three doubles. It's always awkward to read calls, especially when calling with literals because they don't have any semantic meaning attached. People came up with workarounds:

Apparently we need C-style comments to make C++ readable and maintainable [3]. Or preprocessor-magic [4]. Or the workaround called "use a struct and direct-initialize it". Or the "just-so-it-has-a-name" pattern of declared-once, used-once variables:

Interface evolution

When evolving interfaces we have to consider all possible existing invocations. Consider this function:

It is okay to evolve this to the declaration below? Where "okay" means: will all calls provoke compiler errors, such that clients can adjust to the new interface?

No it's not, due to possible calls out there that currently invoke a constructor, but might now invoke a constructor instead.

And we have all lost *hours* of debugging on these issues; scale that with the number of people hitting this problem and you see how relevant this becomes. Yes, we try to make the types really non-cooperative to conversions. On the other hand we *want* interfaces and their invocations to be simple; invocation through is seen as one of the benefits that C++11 brought.

What if we could make this much clearer? This call

is much more of a contract between caller and callee. Long-lived code can make sure that the call remains what it was when it was written. The calling code is proposing to the implementation to check whether the selected overload is the one the call expected. That by itself is just wonderful.

Overload resolution

Overloading works like a charm when it works, and is a curse if the unsuspected happens. (If you question that statement, read this month's thread on ADL and from the LWG reflector.) C++ would be nice to its users if it offered argument names as a crosscheck that code author and compiler agree on the selected overload.

Call: what was the order again?

One of the major sources of errors we see in our code base is due to function calls with wrong parameter order. Sometimes the compiler helps (different types), but very often it doesn't. Take the example from before: without looking it up, do you remember the name of the function from the introduction? You probably did: Gauss. Do you remember the name of the parameters? Likely. But do you remember their position? Compare to . Err, wait - I meant . With this proposal, a good implementation would tell you that you got the order wrong.

Names are far more significant than indices, which is why we use them so much in C++. This is not just cosmetics - this is an actual source of bugs.

Counter-argument: use an IDE. That helps writing code but not reading. And maintenance cost is all about reading code.

But this is not the real thing!

In many other languages, parameter naming allows to specify arguments in arbitrary order, skipping arbitrary defaulted parameters. This enables much more stable interfaces - but it would introduce an ABI dependence on names (see the discussion on R0 of this proposal). So yes, this is not the real thing, far from it. But it's addressing an issue that C and C++ call syntax have since their conception, without turning C++ functions inside out.

Feature Details

Syntax

In function calls, arguments can be named by separating an identifier with from the argument's expression. The use of the identifier has no effect. And as a *Note* we'll add that implementations could verify whether this name is used in any of the redeclarations of the called function.

Again: this proposal does not introduce a new function type, changes nothing in function declarations, is a complete opt-in syntactic sugar. So is the use of names for accessing data members these days (instead of), but wow are member names useful.

Possible diagnostic implementation wrt redeclarations and overridden functions

It would seem reasonable to issue no diagnostic if any declaration of the called function uses the parameter name for the argument it was specified for. If the function is virtual, parameter names used in base classes' declarations could be considered, too. The implementation cost is likely a (possibly lazily built - at least until argument names rule the world) set of known parameter names for each parameter.

Effect on the standard

No effect on the standard is expected: this proposal doesn't change the normative behavior. It might have an effect on the future committee: if the world falls in love with this call syntax, agreeing on parameter names for std2 might be a good idea. Until then, implementations would probably not warn about parameter name mismatches for functions in system headers.

Acknowledgments

Thanks to all the CERN and Fermilab folks who insisted that this matters and provided valuable feedback, criticism and suggestions. Thanks to Ville for suggesting an approach that builds on what is suggested in this paper!

Revision history

Changes compared to revision 0 (P0671R0)

Get the core features with a fairly unintrusive change: new syntax that is ignored for the standard, but enables compilers to emit diagnostics.

References

1. *Bring named parameters in modern C++* by Marco Arena (retrieved on 2017-06-13).

2. *Named arguments* (N4172) by Ehsan Akhgari, Botond Ballo, and its discussion notes from Urbana-Champaign.

3. Typical example of c-style comments to provide a name to an argument.

4. *Boost parameter library* (retrieved on 2017-06-13).